

Python Programming

Day 8: Pandas 기초

Series, DataFrame, Indexing, Modification, CSV, Statistics

Contents

1	학습 목표	2
2	Pandas와 Series	2
2.1	Series	2
2.2	최종 코드: 01_series.py	2
3	DataFrame	3
3.1	최종 코드: 02_dataframe.py	3
4	데이터 선택	4
4.1	열 선택	4
4.2	iloc을 이용한 위치 선택	4
4.3	최종 코드: 03_indexing.py	4
5	데이터 추가와 삭제	5
5.1	최종 코드: 04_modify.py	6
6	CSV 저장과 불러오기	7
6.1	CSV 저장	7
6.2	CSV 불러오기	7
6.3	최종 코드: 05_csv.py	7
7	기초 통계	8
7.1	최종 코드: 06_statistics.py	8
8	Day 8 종합 실습: 학생 성적 관리	9
8.1	프로그램 요구사항	9
8.2	최종 코드: practice/practice_day08.py	9
8.3	실행 결과	10
8.4	프로그램 구조 분석	11
9	실습 파일 목록	11
10	핵심 요약	11

1. 학습 목표

학습 목표

이 장을 마치면 다음 내용을 설명하고 직접 코드로 작성할 수 있다.

- Pandas의 목적과 기본 자료구조 설명하기
- Series를 생성하고 인덱스로 값 선택하기
- 딕셔너리로 DataFrame 생성하기
- 열 선택과 iloc을 이용한 위치 기반 데이터 선택하기
- 새로운 열을 추가하고 drop()으로 열 삭제하기
- to_csv()와 read_csv()로 CSV 저장 및 불러오기
- mean(), sum(), max(), min(), describe() 사용하기

2. Pandas와 Series

Pandas는 Python에서 표 형태의 데이터를 저장하고 분석하기 위한 라이브러리이다. NumPy가 수치 배열 계산에 강하다면, Pandas는 CSV와 같은 실제 데이터 파일을 읽고 원하는 행과 열을 선택하며 통계를 계산하는 작업에 적합하다.

Pandas를 설치하지 않았다면 터미널에서 다음 명령을 한 번 실행한다.

```
pip install pandas
```

Pandas는 일반적으로 pd라는 별칭으로 불러온다.

```
1 import pandas as pd
```

2.1. Series

Series는 값과 인덱스를 함께 가지는 1차원 자료구조이다. 인덱스를 따로 지정하지 않으면 0, 1, 2, ...가 자동으로 생성되지만, 문자열이나 다른 값을 직접 인덱스로 지정할 수도 있다.

```
1 scores = pd.Series(  
2     [90, 85, 100],  
3     index=["Math", "English", "Physics"]  
4 )
```

특정 인덱스의 값은 대괄호 안에 인덱스를 넣어 선택한다.

```
1 print(scores["English"])
```

2.2. 최종 코드: 01_series.py

```
1 import pandas as pd  
2  
3 scores = pd.Series(  
4     [90, 85, 100],  
5     index=["Math", "English", "Physics"]  
6 )  
7  
8 print(scores)
```

```

9 print()
10
11 print(scores["English"])
12 print()
13
14 print(scores[["Math", "Physics"]])
15 print()
16
17 print(scores.values)
18 print(scores.index)

```

실행 결과는 다음과 같다.

```

Math      90
English   85
Physics   100
dtype: int64

```

```
85
```

```

Math      90
Physics   100
dtype: int64

```

```

[ 90  85 100]
Index(['Math', 'English', 'Physics'], dtype='object')

```

핵심 포인트

Series의 인덱스는 0, 1, 2, ...로 고정되지 않는다. 직접 지정한 인덱스를 이용하면 딕셔너리처럼 의미 있는 이름으로 값에 접근할 수 있다.

3. DataFrame

DataFrame은 행과 열을 가지는 2차원 자료구조이다. 엑셀 표와 비슷하며, 여러 개의 Series가 열 방향으로 모인 형태로 이해할 수 있다.

DataFrame은 보통 딕셔너리를 이용하여 만든다. 딕셔너리의 key는 열 이름이 되고, value에 들어 있는 리스트가 해당 열의 데이터가 된다.

Name	Math	English
Kim	90	88
Lee	85	91
Park	100	95

각 열의 길이는 같아야 한다. 길이가 다르면 DataFrame을 만들 수 없다.

3.1. 최종 코드: 02_dataframe.py

```

1 import pandas as pd
2
3 data = {
4     "Name": ["Kim", "Lee", "Park"],
5     "Math": [90, 85, 100],
6     "English": [88, 91, 95]

```

```

7 }
8
9 df = pd.DataFrame(data)
10
11 print(df)
12 print()
13 print(type(df))

```

실행 결과는 다음과 같다.

```

      Name  Math  English
0    Kim     90      88
1    Lee     85      91
2   Park    100      95

```

```
<class 'pandas.core.frame.DataFrame'>
```

핵심 포인트

DataFrame은 열마다 서로 다른 자료형을 가질 수 있다. 예를 들어 이름은 문자열, 점수는 정수, 합격 여부는 불리언으로 저장할 수 있다.

4. 데이터 선택

DataFrame을 만든 뒤에는 필요한 열, 행 또는 하나의 값을 선택할 수 있다.

4.1. 열 선택

하나의 열은 열 이름을 대괄호 안에 넣어 선택한다.

```
1 df["Math"]
```

하나의 열을 선택하면 결과는 Series이다. 여러 열을 선택할 때는 열 이름 목록을 전달한다.

```
1 df[["Math", "English"]]
```

여러 열을 선택하면 결과는 DataFrame이다.

4.2. iloc을 이용한 위치 선택

iloc은 정수 위치를 기준으로 데이터를 선택한다. 위치는 Python 인덱스와 마찬가지로 0부터 시작한다.

표현	의미
df.iloc[0]	첫 번째 행
df.iloc[1]	두 번째 행
df.iloc[1, 2]	두 번째 행, 세 번째 열의 값

df.iloc[1, 2]를 “1행 2열”이라고 표현하면 일반적인 행렬 표기와 혼동될 수 있다. 따라서 위치를 설명할 때는 “행 인덱스 1, 열 인덱스 2” 또는 “두 번째 행, 세 번째 열”이라고 표현하는 것이 정확하다.

4.3. 최종 코드: 03_indexing.py

```

1 import pandas as pd
2
3 data = {
4     "Name": ["Kim", "Lee", "Park"],
5     "Math": [90, 85, 100],
6     "English": [88, 91, 95]
7 }
8
9 df = pd.DataFrame(data)
10
11 print("=== Math Column ===")
12 print(df["Math"])
13 print()
14
15 print("=== Math and English ===")
16 print(df[["Math", "English"]])
17 print()
18
19 print("=== First Row ===")
20 print(df.iloc[0])
21 print()
22
23 print("=== Second Row, Third Column ===")
24 print(df.iloc[1, 2])

```

실행 결과는 다음과 같다.

```
=== Math Column ===
```

```
0    90
1    85
2   100
```

```
Name: Math, dtype: int64
```

```
=== Math and English ===
```

```

      Math  English
0      90      88
1      85      91
2     100      95

```

```
=== First Row ===
```

```

Name      Kim
Math      90
English   88
Name: 0, dtype: object

```

```
=== Second Row, Third Column ===
```

```
91
```

핵심 포인트

열 이름으로 선택할 때는 `df["Column"]`을 사용하고, 실제 저장 위치를 기준으로 행과 열을 선택할 때는 `iloc`을 사용한다.

5. 데이터 추가와 삭제

새로운 열을 추가할 때는 딕셔너리에 항목을 추가하듯이 열 이름에 값을 대입한다.

```
1 df["Physics"] = [95, 80, 90]
```

같은 이름의 열이 이미 존재하면 기존 값이 새로운 값으로 바뀐다.
열을 삭제할 때는 `drop()`을 사용한다.

```
1 df = df.drop(columns=["Math"])
```

`drop()`은 수정된 새로운 DataFrame을 반환하므로, 결과를 다시 `df`에 저장한다.

5.1. 최종 코드: 04_modify.py

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Kim", "Lee", "Park"],
5     "Math": [90, 85, 100]
6 }
7
8 df = pd.DataFrame(data)
9
10 print("=== Original ===")
11 print(df)
12 print()
13
14 df["Physics"] = [95, 80, 90]
15
16 print("=== After Adding Physics ===")
17 print(df)
18 print()
19
20 df = df.drop(columns=["Math"])
21
22 print("=== After Deleting Math ===")
23 print(df)
```

실행 결과는 다음과 같다.

```
=== Original ===
```

	Name	Math
0	Kim	90
1	Lee	85
2	Park	100

```
=== After Adding Physics ===
```

	Name	Math	Physics
0	Kim	90	95
1	Lee	85	80
2	Park	100	90

```
=== After Deleting Math ===
```

	Name	Physics
0	Kim	95
1	Lee	80
2	Park	90

행도 `drop(index=...)`으로 삭제할 수 있다.

```
1 df = df.drop(index=1)
```

핵심 포인트

새로운 열의 데이터 개수는 DataFrame의 행 개수와 같아야 한다. 개수가 다르면 길이가 맞지 않는다는 오류가 발생한다.

6. CSV 저장과 불러오기

CSV는 Comma-Separated Values의 약자로, 쉼표를 이용해 표 데이터를 저장하는 텍스트 파일 형식이다. Pandas를 사용하면 DataFrame을 CSV로 저장하고 다시 읽는 작업을 한 줄씩으로 처리할 수 있다.

6.1. CSV 저장

```
1 df.to_csv("students.csv", index=False)
```

`index=False`를 사용하면 DataFrame의 자동 인덱스가 파일에 별도 열로 저장되지 않는다.

6.2. CSV 불러오기

```
1 loaded = pd.read_csv("students.csv")
```

`read_csv()`의 반환값은 DataFrame이다.

6.3. 최종 코드: 05_csv.py

```
1 import pandas as pd
2
3 data = {
4     "Name": ["Kim", "Lee", "Park"],
5     "Math": [90, 85, 100],
6     "English": [88, 91, 95]
7 }
8
9 df = pd.DataFrame(data)
10
11 df.to_csv("students.csv", index=False)
12
13 loaded = pd.read_csv("students.csv")
14
15 print("=== Loaded CSV ===")
16 print(loaded)
17 print()
18 print(type(loaded))
```

실행 결과는 다음과 같다.

```
=== Loaded CSV ===
```

```
   Name  Math  English
0   Kim    90     88
1   Lee    85     91
2  Park   100     95
```

```
<class 'pandas.core.frame.DataFrame'>
```


핵심 포인트

CSV는 자료형이나 서식을 완전히 보존하는 엑셀 파일과 다르다. 하지만 구조가 단순하고 여러 프로그램에서 쉽게 읽을 수 있어 데이터 분석에서 자주 사용된다.

7. 기초 통계

Pandas의 Series와 DataFrame은 기본적인 통계 함수를 제공한다.

함수	기능
mean()	평균
sum()	합계
max()	최댓값
min()	최솟값
describe()	주요 기술통계 요약

describe()는 데이터 개수, 평균, 표준편차, 최솟값, 사분위수, 최댓값을 한 번에 계산한다.

항목	의미
count	데이터 개수
mean	평균
std	표본 표준편차
min	최솟값
25%	제1사분위수
50%	중앙값
75%	제3사분위수
max	최댓값

7.1. 최종 코드: 06_statistics.py

```

1 import pandas as pd
2
3 data = {
4     "Math": [90, 85, 100, 70, 95]
5 }
6
7 df = pd.DataFrame(data)
8
9 print("Mean:", df["Math"].mean())
10 print("Sum:", df["Math"].sum())
11 print("Max:", df["Math"].max())
12 print("Min:", df["Math"].min())
13
14 print("\n=== Describe ===")
15 print(df.describe())

```

실행 결과는 다음과 같다.

```

Mean: 88.0
Sum: 440
Max: 100
Min: 70

```

```

=== Describe ===
           Math
count      5.000000
mean      88.000000
std       11.510864
min       70.000000
25%      85.000000
50%      90.000000
75%      95.000000
max      100.000000

```

핵심 포인트

통계 함수는 숫자 열을 선택한 뒤 적용하는 것이 기본이다. CSV에서 읽어온 DataFrame에도 같은 방식으로 사용할 수 있다.

8. Day 8 종합 실습: 학생 성적 관리

이번 종합 실습에서는 DataFrame 생성, 데이터 선택, 열 추가 및 삭제, CSV 저장과 불러오기, 기초 통계 기능을 하나의 프로그램에서 연결한다.

8.1. 프로그램 요구사항

1. 네 학생의 이름, 수학 점수, 영어 점수로 DataFrame을 생성한다.
2. 물리 점수 열을 추가한다.
3. 전체 DataFrame과 수학 점수 열을 출력한다.
4. `iloc`을 이용해 두 번째 학생의 정보를 출력한다.
5. 영어 점수 열을 삭제한 뒤 DataFrame을 출력한다.
6. DataFrame을 `student.csv`에 저장하고 다시 불러온다.
7. 불러온 데이터의 수학 평균, 최댓값, 최솟값을 출력한다.
8. `describe()`로 주요 통계 정보를 출력한다.

8.2. 최종 코드: `practice/practice_day08.py`

```

1 import pandas as pd
2
3 data = {
4     "Name": ["Kim", "Lee", "Park", "Choi"],
5     "Math": [90, 85, 100, 95],
6     "English": [88, 91, 95, 87]
7 }
8
9 df = pd.DataFrame(data)
10
11 df["Physics"] = [92, 84, 98, 90]
12
13 print("==== Student Data =====")
14 print(df)
15
16 print()

```

```

17 print(df["Math"])
18
19 print()
20 print(df.iloc[1])
21
22 print()
23 df = df.drop(columns=["English"])
24 print(df)
25
26 df.to_csv("student.csv", index=False)
27 loaded = pd.read_csv("student.csv")
28
29 print()
30 print("Average :", loaded["Math"].mean())
31 print("Maximum :", loaded["Math"].max())
32 print("Minimum :", loaded["Math"].min())
33
34 print()
35 print(loaded.describe())

```

8.3. 실행 결과

==== Student Data ====

	Name	Math	English	Physics
0	Kim	90	88	92
1	Lee	85	91	84
2	Park	100	95	98
3	Choi	95	87	90

```

0    90
1    85
2   100
3    95

```

Name: Math, dtype: int64

```

Name      Lee
Math      85
English   91
Physics   84

```

Name: 1, dtype: object

	Name	Math	Physics
0	Kim	90	92
1	Lee	85	84
2	Park	100	98
3	Choi	95	90

Average : 92.5

Maximum : 100

Minimum : 85

	Math	Physics
count	4.000000	4.000000
mean	92.500000	91.000000
std	6.454972	5.887841

```

min      85.000000  84.000000
25%     88.750000  88.500000
50%     92.500000  91.000000
75%     96.250000  93.500000
max     100.000000  98.000000

```

8.4. 프로그램 구조 분석

기능	사용한 개념
학생 데이터 생성	<code>pd.DataFrame()</code>
물리 점수 추가	새 열 대입
수학 점수 선택	열 인덱싱
두 번째 학생 선택	<code>iloc</code>
영어 점수 삭제	<code>drop()</code>
파일 저장과 불러오기	<code>to_csv()</code> , <code>read_csv()</code>
점수 통계 계산	<code>mean()</code> , <code>max()</code> , <code>min()</code>
전체 통계 확인	<code>describe()</code>

핵심 포인트

CSV를 다시 읽은 뒤에는 `loaded`를 사용하여 통계를 계산했다. 이렇게 하면 저장된 파일이 정상적으로 불러와졌는지까지 함께 확인할 수 있다.

9. 실습 파일 목록

실습

Day 8의 학습 파일은 다음과 같이 관리한다.

- 01.series.py
- 02.dataframe.py
- 03.indexing.py
- 04.modify.py
- 05.csv.py
- 06.statistics.py
- practice/practice_day08.py
- python_day08.tex
- python_day08.pdf

10. 핵심 요약

- Pandas는 표 형태의 데이터를 저장하고 분석하기 위한 Python 라이브러리이다.
- Series는 값과 인덱스를 가지는 1차원 자료구조이다.
- Series의 인덱스는 자동 숫자 인덱스뿐 아니라 문자열 등으로 직접 지정할 수 있다.

- DataFrame은 행과 열을 가지는 2차원 표 형태의 자료구조이다.
- 딕셔너리의 key는 DataFrame의 열 이름이 되고 value는 열 데이터가 된다.
- `df["Column"]`은 하나의 열을 Series로 반환한다.
- `df[["A", "B"]]`는 여러 열을 DataFrame으로 반환한다.
- `iloc`은 0부터 시작하는 정수 위치를 기준으로 데이터를 선택한다.
- `df.iloc[1, 2]`는 두 번째 행, 세 번째 열의 값을 의미한다.
- 새 열은 `df["New"] = values`로 추가하거나 수정한다.
- `drop(columns=[...])`은 지정한 열을 삭제한 새로운 DataFrame을 반환한다.
- `to_csv()`는 DataFrame을 CSV로 저장한다.
- `read_csv()`는 CSV를 DataFrame으로 불러온다.
- `index=False`를 사용하면 자동 인덱스가 CSV에 저장되지 않는다.
- `mean()`, `sum()`, `max()`, `min()`으로 기본 통계를 계산한다.
- `describe()`는 숫자 열의 주요 기술통계를 한 번에 보여준다.
- Pandas의 고급 기능은 실제 데이터 분석이나 프로젝트에서 필요한 시점에 추가로 학습한다.